

Tasks on Jenkins-04

1) Setup Jenkins CI/CD pipeline using freestyle job using Docker containers using below code.

Source Code: <https://github.com/betawins/hiring-app.git>

Stages:

- **Git Clone**
- **SonarQube Integration**
- **Maven Compilation**
- **Nexus Artifactory**
- **Slack Notification**
- **Deploy On tomcat**

Implementation Steps

1. Log in to the Jenkins server.
2. Open Jenkins Dashboard.
3. Click New Item.
4. Enter the job name, for example:
 - hiring-app-freestyle
5. Select Freestyle project.
6. Click OK.
Configure Git Clone
7. Go to Source Code Management.
8. Select Git.
9. Enter the repository URL:
<https://github.com/betawins/hiring-app.git>
10. Set the branch:
*/main
11. If the repository is public, credentials are not required.
12. Click Apply.
Configure Docker/Maven Build

13. Ensure Docker is installed on the Jenkins machine.

14. Verify Docker:

```
docker --version
```

15. Verify that the Jenkins user can execute Docker:

```
sudo usermod -aG docker jenkins
```

16. Restart Jenkins:

```
sudo systemctl restart jenkins
```

17. Go to Build Steps.

18. Select Execute shell.

19. Add the Maven build commands:

```
echo "Running Maven compilation..."
```

```
docker run --rm \
```

```
-v "$WORKSPACE":/app \
```

```
-w /app \
```

```
maven:3.9-eclipse-temurin-17 \
```

```
mvn clean package -DskipTests
```

```
echo "Maven compilation completed."
```

```
ls -lh target/
```

SonarQube Integration

20. Configure SonarQube in Manage Jenkins → System.

21. Add the SonarQube server.

22. Give it a name such as:

```
sonarqube-server
```

23. Configure the SonarQube authentication token if required.

24. Install/configure the SonarQube Scanner for Jenkins plugin.

25. Add another Execute shell build step.

26. Run SonarQube analysis using the Maven container:

```
docker run --rm \
-v "$WORKSPACE":/app \
-w /app \
maven:3.9-eclipse-temurin-17 \
mvn org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \
-Dsonar.projectKey=hiring-app \
-Dsonar.projectName=HiringApp \
-Dsonar.host.url=http://<SONARQUBE-IP>:9000
```

27. Replace <SONARQUBE-IP> with the actual SonarQube server IP.

Nexus Artifactory

28. Configure Nexus credentials in Jenkins → Credentials.

29. Create the credential ID:

nexus-cred

30. Create/configure the Nexus repository:

hiring_app

31. Install the Nexus Artifact Uploader plugin.

32. Add the Nexus upload configuration.

33. Upload:

target/hiring.war

34. Use:

Group ID : in.javahome

Artifact ID : hiring

Version : 0.1

Repository : hiring_app

Slack Notification

35. Configure the Slack integration in Jenkins.

36. Add the Slack credential.

37. Use the required Slack channel.

38. Add a post-build Slack notification.

39. Configure the notification for:

- Success
- Failure

Tomcat Deployment

40. Install/configure the Deploy to container plugin.

41. Add Tomcat credentials in Jenkins.

42. Configure the Tomcat server URL, for example:

`http://<TOMCAT-IP>:9090`

43. Configure the Tomcat deployment context:

`hiring`

44. Select the generated WAR:

`target/hiring.war`

45. Click Apply.

46. Click Save.

47. Click Build Now.

48. Verify all six stages in the Jenkins console output.

49. After successful deployment, verify the application in Tomcat:

`http://<TOMCAT-IP>:9090/hiring/`

us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#Instances:v=3;instanceState=running

EC2 > Instances

Instances (4) Info

Instance state: Running

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone
sonarqube-server	i-0c7aed728b0b61b6c	Running	m7i-flex.large	3/3 checks passed	us-east-1c
Nexus-server	i-09db8c351bc4e2562	Running	c7i-flex.large	3/3 checks passed	us-east-1c
tomcat-server	i-0b754da8c20aa2aba	Running	t3.small	3/3 checks passed	us-east-1c
jenkins-new-server	i-037feae4aaf9bec10	Running	c7i-flex.large	3/3 checks passed	us-east-1b

All instances

```
MINGW64/c:/Users/91738/Downloads
Total
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
Preparing
Running scriptlet: jenkins-2.580.1-1.noarch
Installing : jenkins-2.580.1-1.noarch
Running scriptlet: jenkins-2.580.1-1.noarch
Verifying : jenkins-2.580.1-1.noarch

Installed:
jenkins-2.580.1-1.noarch

Complete!
[root@ip-172-31-12-214 ~]# rpm -qa | grep jenkins
jenkins-2.580.1-1.noarch
[root@ip-172-31-12-214 ~]# sudo systemctl enable jenkins
Created symlink /etc/systemd/system/multi-user.target.wants/jenkins.service -> /usr/lib/systemd/system/jenkins.service.
[root@ip-172-31-12-214 ~]# sudo systemctl start jenkins
[root@ip-172-31-12-214 ~]# sudo systemctl status jenkins
jenkins.service - Jenkins Continuous Integration Server
Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
Active: active (running) since Mon 2026-10-05 09:36:19 UTC; 12s ago
Main PID: 27364 (java)
Tasks: 51 (limit: 4543)
Memory: 649.5M
CPU: 12.763s
CGroup: /system.slice/jenkins.service
└─27364 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080

Oct 05 09:36:16 ip-172-31-12-214.ec2.internal jenkins[27364]: [LF]> This may also be found at: /var/lib/jenkins/secrets/initialAdminPassword
Oct 05 09:36:16 ip-172-31-12-214.ec2.internal jenkins[27364]: [LF]>
Oct 05 09:36:16 ip-172-31-12-214.ec2.internal jenkins[27364]: [LF]> *****
Oct 05 09:36:16 ip-172-31-12-214.ec2.internal jenkins[27364]: [LF]> *****
Oct 05 09:36:19 ip-172-31-12-214.ec2.internal jenkins[27364]: 2026-10-05 09:36:19.746+0000 [id=40] INFO jenkins.InitReactorRunner$1:onAttained: Completed in
Oct 05 09:36:19 ip-172-31-12-214.ec2.internal jenkins[27364]: 2026-10-05 09:36:19.763+0000 [id=30] INFO hudson.lifecycle.Lifecycle$onReady: Jenkins is fully
Oct 05 09:36:19 ip-172-31-12-214.ec2.internal systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server.
Oct 05 09:36:19 ip-172-31-12-214.ec2.internal jenkins[27364]: 2026-10-05 09:36:19.953+0000 [id=56] INFO h.m.DownloadService$Downloadable#load: Obtained the u
Oct 05 09:36:19 ip-172-31-12-214.ec2.internal jenkins[27364]: 2026-10-05 09:36:19.955+0000 [id=56] INFO hudson.util.Retrier#start: Performed the action check

[root@ip-172-31-12-214 ~]# sudo ss -ltnp | grep 8080
LISTEN 0 50 *:8080 *: * users:([ "java",pid=27364,fd=9])
[root@ip-172-31-12-214 ~]# cat /var/lib/jenkins/secrets/initialAdminPassword
bf8a6e9c9051464eb99fbdedff80af28
[root@ip-172-31-12-214 ~]# cd /opt
```

```
root@ip-172-31-12-214:/var/lib/jenkins/workspace/hiring_app/target
[root@ip-172-31-12-214 opt]# wget https://d1cdn.apache.org/maven/maven-3/3.10.0/binaries/apache-maven-3.10.0-bin.tar.gz
--2026-10-05 10:34:08-- https://d1cdn.apache.org/maven/maven-3/3.10.0/binaries/apache-maven-3.10.0-bin.tar.gz
Resolving d1cdn.apache.org (d1cdn.apache.org)... 151.101.2.132, 2a04:4e42::644
Connecting to d1cdn.apache.org (d1cdn.apache.org)|151.101.2.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 9979885 (9.5M) [application/x-gzip]
Saving to: 'apache-maven-3.10.0-bin.tar.gz'

apache-maven-3.10.0-bin.tar.gz 100%[=====] 9.52M --.-KB/s in 0.02s

2026-10-05 10:34:09 (403 MB/s) - 'apache-maven-3.10.0-bin.tar.gz' saved [9979885/9979885]

[root@ip-172-31-12-214 opt]# ls
apache-maven-3.10.0-bin.tar.gz aws sonar-scanner sonar-scanner-cli-7.0.0.4796-linux-x64.zip
[root@ip-172-31-12-214 opt]# tar xvf apache-maven-3.10.0-bin.tar.gz
apache-maven-3.10.0/README.txt
apache-maven-3.10.0/LICENSE
apache-maven-3.10.0/NOTICE
apache-maven-3.10.0/lib/
apache-maven-3.10.0/lib/aopalliance.license
apache-maven-3.10.0/lib/asm.license
apache-maven-3.10.0/lib/commons-cli.license
apache-maven-3.10.0/lib/commons-codec.license
apache-maven-3.10.0/lib/error_prone_annotations.license
apache-maven-3.10.0/lib/failureaccess.license
apache-maven-3.10.0/lib/gson.license
apache-maven-3.10.0/lib/guava.license
apache-maven-3.10.0/lib/guice.license
apache-maven-3.10.0/lib/httpcore.license
apache-maven-3.10.0/lib/jansi-core.license
apache-maven-3.10.0/lib/javax.annotation-api.license
apache-maven-3.10.0/lib/javax.inject.license
apache-maven-3.10.0/lib/jcl-over-slf4j.license
apache-maven-3.10.0/lib/jline-native.license
apache-maven-3.10.0/lib/jline-terminal-ffm.license
apache-maven-3.10.0/lib/jline-terminal-jni.license
apache-maven-3.10.0/lib/jline-terminal.license
apache-maven-3.10.0/lib/jspecify.license
apache-maven-3.10.0/lib/org.eclipse.sisu.inject.license
apache-maven-3.10.0/lib/org.eclipse.sisu.plexus.license
apache-maven-3.10.0/lib/plexus-cipher.license
apache-maven-3.10.0/lib/plexus-component-annotations.license
apache-maven-3.10.0/lib/plexus-interpolation.license
apache-maven-3.10.0/lib/plexus-sec-dispatcher.license
apache-maven-3.10.0/lib/plexus-utils.license
apache-maven-3.10.0/lib/slf4j-api.license
apache-maven-3.10.0/boot/
```

```
root@ip-172-31-12-214:/var/lib/jenkins/workspace/hiring_app/target
apache-maven-3.10.0/lib/maven-resolver-provider-3.10.0.jar
apache-maven-3.10.0/lib/maven-resolver-util-2.0.24.jar
apache-maven-3.10.0/lib/plexus-utils-3.6.2.jar
apache-maven-3.10.0/lib/error_prone_annotations-2.48.0.jar
apache-maven-3.10.0/lib/maven-slf4j-provider-3.10.0.jar
apache-maven-3.10.0/lib/wagon-http-shared-3.5.3.jar
apache-maven-3.10.0/lib/httpclient-4.5.14.jar
apache-maven-3.10.0/lib/jansi-core-3.10.1.jar
apache-maven-3.10.0/lib/maven-resolver-impl-2.0.24.jar
apache-maven-3.10.0/lib/maven-settings-3.10.0.jar
apache-maven-3.10.0/lib/maven-embedder-3.10.0.jar
apache-maven-3.10.0/lib/commons-cli-1.11.0.jar
[root@ip-172-31-12-214 opt]# ls
apache-maven-3.10.0 apache-maven-3.10.0-bin.tar.gz aws sonar-scanner sonar-scanner-cli-7.0.0.4796-linux-x64.zip
[root@ip-172-31-12-214 opt]# mv apache-maven-3.10.0 apache-maven
[root@ip-172-31-12-214 opt]# ls
apache-maven apache-maven-3.10.0-bin.tar.gz aws sonar-scanner sonar-scanner-cli-7.0.0.4796-linux-x64.zip
[root@ip-172-31-12-214 opt]# cd
[root@ip-172-31-12-214 ~]# cd /opt/apache-maven/bin/
[root@ip-172-31-12-214 bin]# ls
m2.conf mvn mvn.cmd mvnDebug.cmd mvnyjp
[root@ip-172-31-12-214 bin]# pwd
/opt/apache-maven/bin
[root@ip-172-31-12-214 bin]# mvn --version
-bash: mvn: command not found
[root@ip-172-31-12-214 bin]# cd
[root@ip-172-31-12-214 ~]# vi .bash_profile
[root@ip-172-31-12-214 ~]# source ~/.bash_profile
-bash: MVN_HOME: command not found
[root@ip-172-31-12-214 ~]# vi .bash_profile
[root@ip-172-31-12-214 ~]# vi .bash_profile
[root@ip-172-31-12-214 ~]# source ~/.bash_profile
[root@ip-172-31-12-214 ~]# mvn --version
Apache Maven 3.10.0 (c43a36b8d67be7e0805a411bc0898af1a51f5472)
Maven home: /opt/apache-maven
Java version: 21.0.12.1, vendor: Amazon.com Inc., runtime: /usr/lib/jvm/java-21-amazon-corretto.x86_64
Default locale: en, platform encoding: UTF-8, time zone: UTC
OS name: "linux", version: "6.18.51-120.163.amzn2023.x86_64", arch: "amd64", family: "unix"
[root@ip-172-31-12-214 ~]# cd /var/lib/jenkins/workspace/
[root@ip-172-31-12-214 workspace]# ls
hiring_app
[root@ip-172-31-12-214 workspace]# cd hiring_app/
[root@ip-172-31-12-214 hiring_app]# ls
Dockerfile Jenkinsfile README.md 'Untitled Diagram.drawio' jenkinsfile-cicd pom.xml src target
[root@ip-172-31-12-214 hiring_app]# cd target/
[root@ip-172-31-12-214 target]# ls
hiring_hiring.war maven-archiver
[root@ip-172-31-12-214 target]#
```

```
MINGW64/c/Users/91738/Downloads
root | localhost |
5 rows in set (0.00 sec)

mysql> SHOW GRANTS FOR 'sonar'@'%';
+-----+
| Grants for sonar@% |
+-----+
| GRANT USAGE ON *.* TO 'sonar'@'%' |
| GRANT ALL PRIVILEGES ON 'sonar'.* TO 'sonar'@'%' |
+-----+
2 rows in set (0.00 sec)

mysql> exit;
Bye
[root@ip-172-31-20-90 ~]# sudo sysctl -w vm.max_map_count=262144
vm.max_map_count = 262144
[root@ip-172-31-20-90 ~]# sudo vi /etc/sysctl.conf
[root@ip-172-31-20-90 ~]# sudo sysctl -p
vm.max_map_count = 262144
[root@ip-172-31-20-90 ~]# docker pull sonarqube:latest
latest: Pulling from library/sonarqube
a69ef8ec184e: Pull complete
96fe3bd0148e: Pull complete
54929e68a7af: Pull complete
4f4fb700af54: Pull complete
0a66c41acdb: Pull complete
065adb12f4f5: Pull complete
03217b7e6be2: Pull complete
Digest: sha256:c0f1160bccfa435db4168c2d7df69af3c3ea7bfeb87395613014048fb33a68c1
Status: Downloaded newer image for sonarqube:latest
docker.io/library/sonarqube:latest
[root@ip-172-31-20-90 ~]# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                    NAMES
7b6cdd513824   mysql:5.7     "docker-entrypoint.s..." 8 minutes ago  Up 8 minutes  3306/tcp, 33060/tcp      mysql57
[root@ip-172-31-20-90 ~]# docker images | grep sonarqube
sonarqube      latest        01676bb7354b   3 days ago    1.44GB
[root@ip-172-31-20-90 ~]# docker run -d \
--name sonarqube \
--network sonar-network \
-p 9002:9000 \
-e SONARQUBE_JDBC_USERNAME=sonar \
-e SONARQUBE_JDBC_PASSWORD=sonar@123 \
-e SONARQUBE_JDBC_URL="jdbc:mysql://mysql57:3306/sonar?useUnicode=true&characterEncoding=utf8&useSSL=false&allowPublicKeyRetrieval=true" \
--restart unless-stopped \
sonarqube:latest
f3a7504fd8a68160a7cfdc8b1a1789a35425093615458a0489885993d7a2232e
[root@ip-172-31-20-90 ~]# docker ps
```

hiring_app - Jenkins | hiring_app Config - Jenkins | Tools - Manage Jenkins - Jenkins | HTTP Status 404 - Not Found

← → ↻ 🔒 Not secure 3.93.154.157:8080/job/hiring_app/ ☆ Incognito New Chrome available

Jenkins / hiring_app

Status

Changes

Workspace

Build Now

Delete Project

Configure

Rename

SonarQube

Builds >

Filter

Today

#6 2:31 AM

hiring_app

SonarQube

SonarQube Quality Gate

HiringApp Passed

server-side processing: Success

Permalinks

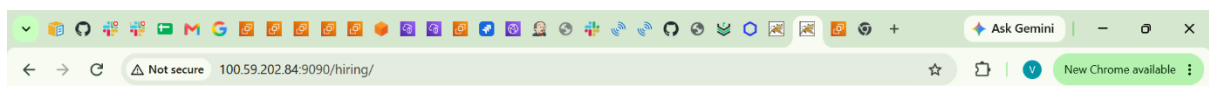
- Last build (#6), 7 hr 16 min ago
- Last stable build (#6), 7 hr 16 min ago
- Last successful build (#6), 7 hr 16 min ago
- Last failed build (#3), 7 hr 48 min ago
- Last unsuccessful build (#3), 7 hr 48 min ago
- Last completed build (#6), 7 hr 16 min ago

3.93.154.157:8080/job/hiring_app/lastSuccessfulBuild/


```
MINGW64/c/Users/91738/Downloads
https://aws.amazon.com/linux/amazon-linux-2023

Last login: Tue Oct 6 04:34:49 2026 from 49.205.231.217
[ec2-user@ip-172-31-19-67 ~]$
[ec2-user@ip-172-31-19-67 ~]$ sudo su -
Last login: Tue Oct 6 04:35:12 UTC 2026 on pts/1
[root@ip-172-31-19-67 ~]# cd /opt/
[root@ip-172-31-19-67 opt]# cd tomcat/
[root@ip-172-31-19-67 tomcat]# ls
BUILDING.txt  LICENSE  README.md  RUNNING.txt  conf  logs  webapps
CONTRIBUTING.md  NOTICE  RELEASE-NOTES  bin  lib  temp  work
[root@ip-172-31-19-67 tomcat]# cd bin/
[root@ip-172-31-19-67 bin]# ls
bootstrap.jar  configtest.sh  shutdown.sh
catalina-tasks.xml  daemon.sh  startup.bat
catalina.bat  digest.sh  startup.sh
catalina.sh  digest.sh  tomcat-juli.jar
ciphers.bat  makebase.bat  tomcat-native.tar.gz
ciphers.sh  makebase.sh  tool-wrapper.bat
commons-daemon-native.tar.gz  setclasspath.bat  tool-wrapper.sh
commons-daemon.jar  setclasspath.sh  version.bat
configtest.bat  shutdown.bat  version.sh
[root@ip-172-31-19-67 bin]# ./shutdown.sh
Using CATALINA_BASE:   /opt/tomcat
Using CATALINA_HOME:   /opt/tomcat
Using CATALINA_TMPDIR: /opt/tomcat/temp
Using JRE_HOME:        /usr
Using CLASSPATH:       /opt/tomcat/bin/bootstrap.jar:/opt/tomcat/bin/tomcat-juli.jar
NOTE: Picked up JDK_JAVA_OPTIONS:  --add-opens=java.base/java.lang=ALL-UNNAMED --add-opens=java.base/java.lang.invoke=ALL-UNNAMED --add-opens=java.base/java.lang.reflect=ALL-UNNAMED --add-opens=java.base/java.io=ALL-UNNAMED --add-opens=java.base/java.util=ALL-UNNAMED --add-opens=java.base/java.util.concurrent=ALL-UNNAMED --add-opens=java.rmi/sun.rmi.transport=ALL-UNNAMED
Using CATALINA_BASE:   /opt/tomcat
Using CATALINA_HOME:   /opt/tomcat
Using CATALINA_TMPDIR: /opt/tomcat/temp
Using JRE_HOME:        /usr
Using CLASSPATH:       /opt/tomcat/bin/bootstrap.jar:/opt/tomcat/bin/tomcat-juli.jar
Using CATALINA_OPTS:
Tomcat started.
[root@ip-172-31-19-67 bin]# client_loop: send disconnect: Connection reset by peer

91738@IDEAPAD MINGW64 ~/Downloads
$
```



2) Setup a Jenkins CICD pipeline using Declarative pipeline using feature-1.1 branch.

Source Code:

https://github.com/betawins/sabear_simplecutomerapp/tree/feature-1.1
stages:

- Git Clone
- SonarQube Integration
- Maven Compilation

- **Nexus Artifactory**
- **Slack Notification**
- **Deploy On tomcat**

Implementation Steps

1. Open Jenkins Dashboard.
2. Click New Item.
3. Enter:
simplecustomerapp-de-pipeline
4. Select Pipeline.
5. Click OK.
6. Scroll to the Pipeline section.
7. Select Pipeline script.
8. Use a Declarative Pipeline structure.

```
pipeline {
```

```
    agent any
```

```
    tools {
```

```
        maven 'MVN_HOME'
```

```
    }
```

```
    environment {
```

```
        // =====
```

```
        // APPLICATION
```

```
        // =====
```

```
        APP_NAME = 'SimpleCustomerApp'
```

```
// =====  
  
// GIT  
  
// =====  
  
GIT_URL = 'https://github.com/betawins/sabear_simplecutomerapp.git'  
GIT_BRANCH = 'feature-1.1'  
  
// =====  
  
// SONARQUBE  
  
// =====  
  
SONAR_SERVER = 'sonarqube-server'  
SONAR_URL = 'http://52.90.154.40:9002'  
  
// =====  
  
// NEXUS  
  
// =====  
  
NEXUS_URL = '98.93.245.155:8081'  
NEXUS_REPOSITORY = 'samplecustomer_app'  
NEXUS_CREDENTIALS = 'nexus-cred'  
  
// =====  
  
// TOMCAT  
  
// =====  
  
TOMCAT_URL = 'http://100.59.202.84:9090'  
TOMCAT_CREDENTIALS = 'tomcat-cred'  
TOMCAT_CONTEXT = 'SimpleCustomerApp'
```

```

// =====

// SLACK

// =====

SLACK_CHANNEL = '#all-jenkins-bot'
SLACK_CREDENTIALS = 'slack-cred'


// =====

// MAVEN ARTIFACT

// =====

GROUP_ID = 'com.javatpoint'
ARTIFACT_ID = 'SimpleCustomerApp'
APP_VERSION = "${BUILD_NUMBER}-SNAPSHOT"
WAR_FILE = "target/SimpleCustomerApp-${BUILD_NUMBER}-SNAPSHOT.war"
}

stages {

// =====

// 1. GIT CLONE

// =====

stage('Git Clone') {

    steps {

```

```
    echo ""
'''

=====

STAGE 1: GIT CLONE

=====

'''

deleteDir()

git(
    branch: "${GIT_BRANCH}",
    url: "${GIT_URL}"
)

sh ""
    set -e

    echo "Git clone completed successfully."

    echo ""
    echo "Current branch:"
    git branch --show-current

    echo ""
    echo "Latest commit:"
```

```
git log -1 --oneline
```

```
echo ""
```

```
echo "Workspace:"
```

```
pwd
```

```
echo ""
```

```
echo "Repository files:"
```

```
ls -la
```

```
""
```

```
}
```

```
}
```

```
// =====
```

```
// 2. PREPARE BUILD POM
```

```
// =====
```

```
stage('Prepare Build POM') {
```

```
  steps {
```

```
    echo ""
```

```
    ...
```

```
=====
```

```
STAGE 2: PREPARE BUILD POM
```

```
=====
```

'''

```
sh '''
```

```
    set -e
```

```
    echo "Original pom.xml:"
```

```
    cat pom.xml
```

```
    echo ""
```

```
    echo "Creating Jenkins temporary POM..."
```

```
    cp pom.xml jenkins-build-pom.xml
```

```
    sed -i "s#<version>\\${BUILD_NUMBER}-  
    SNAPSHOT</version>#<version>${BUILD_NUMBER}-SNAPSHOT</version>#g"  
    jenkins-build-pom.xml
```

```
    echo ""
```

```
    echo "Jenkins temporary POM:"
```

```
    cat jenkins-build-pom.xml
```

```
    echo ""
```

```
    echo "Validating Jenkins POM..."
```

```
    mvn -f jenkins-build-pom.xml validate
```

```
    echo ""
```

```
    echo "Temporary POM validation successful."
```

```
'''
```

```
}  
}
```

```
// =====  
// 3. MAVEN COMPILATION  
// =====
```

```
stage('Maven Compilation') {
```

```
    steps {
```

```
        echo ""
```

```
    }
```

```
=====
```

```
STAGE 3: MAVEN COMPILATION
```

```
=====
```

```
'''
```

```
    sh '''
```

```
        set -e
```

```
        echo "Maven version:"
```

```
        mvn --version
```

```
    '''
```

```
echo "Java version:"
```

```
java -version
```

```
echo ""
```

```
echo "Removing old target directory..."
```

```
rm -rf target
```

```
echo ""
```

```
echo "Building application..."
```

```
mvn -f jenkins-build-pom.xml clean package -DskipTests
```

```
echo ""
```

```
echo "Maven build completed successfully."
```

```
echo ""
```

```
echo "Target directory:"
```

```
ls -lh target/
```

```
echo ""
```

```
echo "WAR files:"
```

```
find target -maxdepth 1 -type f -name "*.war" -ls
```

```
""
```

```
}
```

```
}
```

```
//=====
```

```

// 4. SONARQUBE INTEGRATION

// =====

stage('SonarQube Integration') {

    steps {

        echo ""

        ""

        =====

        STAGE 4: SONARQUBE ANALYSIS

        =====

        ""

        withSonarQubeEnv("${SONAR_SERVER}") {

            sh ""

            set -e

            echo "Running SonarQube analysis..."

            echo ""

            echo "SonarQube URL:"

            echo "${SONAR_URL}"

            echo ""

```

```
echo "Executing Sonar Maven Plugin..."
```

```
mvn \
org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \
-Dsonar.projectKey=SimpleCustomerAppKey \
-Dsonar.projectName=SimpleCustomerApp \
-Dsonar.projectVersion=${BUILD_NUMBER} \
-Dsonar.host.url=${SONAR_URL}
```

```
echo ""
echo "SonarQube analysis completed successfully."
```

```
""
```

```
}
```

```
}
```

```
}
```

```
// =====
```

```
// 5. VERIFY WAR
```

```
// =====
```

```
stage('Verify WAR') {
```

```
  steps {
```

```
    echo ""
```

```
    ...
```

```
=====
```

STAGE 5: VERIFY WAR

=====

'''

sh '''

set -e

echo "Expected WAR:"

echo "\${WAR_FILE}"

if [! -f "\${WAR_FILE}"]; then

echo ""

echo "ERROR: WAR file was not created."

echo ""

echo "Files inside target:"

ls -lah target/

exit 1

fi

echo ""

echo "WAR file found successfully."

ls -lh "\${WAR_FILE}"

```

        echo ""

        echo "WAR contents:"

        unzip -l "${WAR_FILE}"

        echo ""

        echo "Checking application files..."

        unzip -l "${WAR_FILE}" | grep -E
        "index.html|Welcome.jsp|Customer.jsp|WEB-INF" || true

        echo ""

        echo "WAR verification completed."
    ""
}

}

// =====
// 6. NEXUS ARTIFACT UPLOAD
// =====

stage('Nexus Artifactory') {

    steps {

        echo ""
    ""

    =====

```

STAGE 6: NEXUS ARTIFACT UPLOAD

=====

'''

sh '''

set -e

echo "Nexus URL:"

echo "\${NEXUS_URL}"

echo ""

echo "Nexus repository:"

echo "\${NEXUS_REPOSITORY}"

echo ""

echo "Artifact:"

echo "\${WAR_FILE}"

echo ""

echo "Checking WAR..."

ls -lh "\${WAR_FILE}"

'''

nexusArtifactUploader(

nexusVersion: 'nexus3',

protocol: 'http',

nexusUrl: "\${NEXUS_URL}",

groupId: "\${GROUP_ID}",

version: "\${APP_VERSION}",

repository: "\${NEXUS_REPOSITORY}",

credentialsId: "\${NEXUS_CREDENTIALS}",

artifacts: [

[

artifactId: "\${ARTIFACT_ID}",

classifier: "",

file: "\${WAR_FILE}",

type: 'war'

]

]

)

echo ""

'''

=====

NEXUS UPLOAD SUCCESSFUL

=====

'''

}

}

// =====

// 7. SLACK NOTIFICATION

// =====

stage('Slack Notification') {

steps {

echo '''

'''

=====

STAGE 7: SLACK NOTIFICATION

=====

'''

slackSend(

channel: "\${SLACK_CHANNEL}",

color: 'good',

tokenCredentialId: "\${SLACK_CREDENTIALS}",

botUser: true,

message: ""

``

SUCCESS: SimpleCustomerApp Pipeline

Application : \${APP_NAME}

Job : \${JOB_NAME}

Build : #\${BUILD_NUMBER}

Branch : \${GIT_BRANCH}

SonarQube : \${SONAR_URL}

Nexus : http://\${NEXUS_URL}

Repository : \${NEXUS_REPOSITORY}

Tomcat : \${TOMCAT_URL}

Context : \${TOMCAT_CONTEXT}

WAR : \${WAR_FILE}

Jenkins:

\${BUILD_URL}

```
""
```

```
)
```

```
}
```

```
}
```

```
// =====
```

```
// 8. DEPLOY ON TOMCAT
```

```
// =====
```

```
stage('Deploy On Tomcat') {
```

```
  steps {
```

```
    echo ""
```

```
    ""
```

```
=====
```

```
STAGE 8: DEPLOY ON TOMCAT
```

```
=====
```

```
""
```

```
echo "Tomcat URL: ${TOMCAT_URL}"
```

```
echo "WAR file: ${WAR_FILE}"
```

```
deploy(
```

```

adapters: [

    tomcat9(

        credentialsId: "${TOMCAT_CREDENTIALS}",

        url: "${TOMCAT_URL}",

        path: "

    )

],

contextPath: "${TOMCAT_CONTEXT}",

war: "${WAR_FILE}"
)

echo ""

...

=====

TOMCAT DEPLOYMENT COMPLETED

=====

""

}

```

```
}
```

```
// =====
```

```
// 9. VERIFY TOMCAT DEPLOYMENT
```

```
// =====
```

```
stage('Verify Tomcat Deployment') {
```

```
    steps {
```

```
        echo ""
```

```
    ""
```

```
=====
```

```
STAGE 9: VERIFY TOMCAT DEPLOYMENT
```

```
=====
```

```
""
```

```
    sh ""
```

```
        set +e
```

```
        echo "Checking Tomcat Manager..."
```

```
        curl -u deployer:deployer \
```

```
        "${TOMCAT_URL}/manager/text/list"
```

```
echo ""
```

```
echo "Checking SimpleCustomerApp..."
```

```
HTTP_STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
"${TOMCAT_URL}/${TOMCAT_CONTEXT}/")
```

```
echo ""
```

```
echo "Application HTTP status: ${HTTP_STATUS}"
```

```
echo ""
```

```
if [ "${HTTP_STATUS}" = "200" ]; then
```

```
    echo "SimpleCustomerApp is responding successfully."
```

```
elif [ "${HTTP_STATUS}" = "302" ]; then
```

```
    echo "SimpleCustomerApp is deployed and returned HTTP 302."
```

```
elif [ "${HTTP_STATUS}" = "404" ]; then
```

```
    echo "Application is deployed but returned HTTP 404."
```

```
echo "Checking Tomcat Manager application list..."
```

```
curl -u deployer:deployer \
"${TOMCAT_URL}/manager/text/list"
```

```
        echo ""

        echo "IMPORTANT:"
        echo "A 404 can mean the WAR is deployed but the WAR does not
contain"

        echo "a valid welcome/resource path."

    else

        echo "Application returned HTTP ${HTTP_STATUS}."

    fi

    echo ""
    echo "Tomcat verification completed."
}

}

}

// =====
// POST ACTIONS
// =====

post {

    success {

        echo ""
```

'''

=====

PIPELINE SUCCESS

=====

'''

echo "Application deployment completed."

echo ""

echo "Application URL:"

echo "\${TOMCAT_URL}/\${TOMCAT_CONTEXT}/"

slackSend(

channel: "\${SLACK_CHANNEL}",

color: 'good',

tokenCredentialId: "\${SLACK_CREDENTIALS}",

botUser: true,

message: ""

'''

PIPELINE SUCCESS

Application : \${APP_NAME}

Job : \${JOB_NAME}

Build : #\${BUILD_NUMBER}

Branch : \${GIT_BRANCH}

SonarQube:

\${SONAR_URL}

Nexus:

http://\${NEXUS_URL}

Tomcat:

\${TOMCAT_URL}/\${TOMCAT_CONTEXT}/

Jenkins:

\${BUILD_URL}

""""

)

}

failure {

echo ""

...

=====

PIPELINE FAILED

=====

'''

echo "Please check Jenkins Console Output."

slackSend(

channel: "\${SLACK_CHANNEL}",

color: 'danger',

tokenCredentialId: "\${SLACK_CREDENTIALS}",

botUser: true,

message: ""

'''

PIPELINE FAILED

Application : \${APP_NAME}

Job : \${JOB_NAME}

Build : #\${BUILD_NUMBER}

Branch : \${GIT_BRANCH}

Please check Jenkins Console Output.

Jenkins:

```
${BUILD_URL}
```

```
""""
```

```
)
```

```
}
```

```
always {
```

```
    echo ""
```

```
""
```

```
=====
```

```
PIPELINE EXECUTION COMPLETED
```

```
=====
```

```
""
```

```
}
```

```
}
```

```
}
```

9. Configure Git with:

Repository:

https://github.com/betawins/sabear_simplecutomerapp.git

Branch:

*/feature-1.1

10. Configure the Maven tool in Manage Jenkins → Tools.

11. Configure SonarQube.

12. Configure Nexus credentials.

13. Configure Slack credentials.

14. Configure Tomcat credentials.

15. Implement the following stages:

pipeline

└─ Git Clone

└─ Maven Compilation

└─ SonarQube Integration

└─ Nexus Artifactory

└─ Slack Notification

└─ Deploy On Tomcat

16. The pipeline should use the feature-1.1 branch.

17. Maven should generate the WAR file.

18. Verify the generated WAR before uploading it to Nexus.

19. Upload the WAR to the hiring_app or required Nexus repository.

20. Send a Slack success notification.

21. Deploy the WAR to Tomcat.

22. Click Save.

23. Click Build Now.

24. Verify the Jenkins console output.

25. Verify the deployed application in Tomcat.

simplecustomerapp-de-pipeline x hiring_app Config - Jenkins x Tools - Manage Jenkins - Jenkins x HTTP Status 404 - Not Found x +

Not secure 3.93.154.157:8080/job/simplecustomerapp-de-pipeline/configure

Jenkins / simplecustomerapp-de-pi... / Configure

Configure

- General
- Triggers
- Pipeline
- Advanced

Script ?

```
1 pipeline {
2
3   agent any
4
5   tools {
6     maven 'MVN_HOME'
7   }
8
9   environment {
10
11     // =====
12     // APPLICATION
13     // =====
14
15     APP_NAME = 'SimpleCustomerApp'

```

☒ Use Groovy Sandbox ?

Save Apply

33° 15:07 06-10-2026

simplecustomerapp-de-pipeline x hiring_app Config - Jenkins x Tools - Manage Jenkins - Jenkins x HTTP Status 404 - Not Found x +

Not secure 3.93.154.157:8080/job/simplecustomerapp-de-pipeline/

Jenkins / simplecustomerapp-de-pi...

- <> Changes
- ▷ Build Now
- 🗑 Delete Pipeline
- ⚙ Configure
- ✎ Rename
- 🔊 SonarQube
- 📁 Stages
- ❓ Pipeline Syntax

SonarQube Quality Gate

SimpleCustomerApp **Passed**

server-side processing: **Success**

Permalinks

- Last build (#17), 3 hr 44 min ago
- Last stable build (#17), 3 hr 44 min ago
- Last successful build (#17), 3 hr 44 min ago
- Last failed build (#16), 3 hr 45 min ago
- Last unsuccessful build (#16), 3 hr 45 min ago
- Last completed build (#17), 3 hr 44 min ago

Builds >

Q Filter /

Today

- ✓ #17 5:52 AM
- ✗ #16 5:51 AM

33° 15:07 06-10-2026

simplecustomerapp-de-pipeline x hiring_app Config - Jenkins x Tools - Manage Jenkins - Jenkins x HTTP Status 404 - Not Found x +

Not secure 3.93.154.157:8080/job/simplecustomerapp-de-pipeline/17/console

Jenkins / simplecustomerapp-de-pi... #17

```
Application URL:
[Pipeline] echo
http://100.59.202.84:9090/SimpleCustomerApp/
[Pipeline] slackSend
Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: jenkins-bot, channel: #all-jenkins-bot, color: good, botUser: true, tokenCredentialId: slack-cred, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Jenkins 2.580.1

33°

Ask Gemini

Not secure 98.93.245.155:8081/#browse/browse:samplecustomer_app:com%2Fjavatpoint%2FSimpleCustomerApp%2F15-SNAPSHOT

sonatype nexus repository Community Edition

Search components or CVEs...

Browse / samplecustomer_app

HTML View

Advanced

com

javatpoint

SimpleCustomerApp

1.0.0-SNAPSHOT

15-SNAPSHOT

17-SNAPSHOT

17-20261006.055306-1

maven-metadata.xml

maven-metadata.xml.md5

maven-metadata.xml.sha1

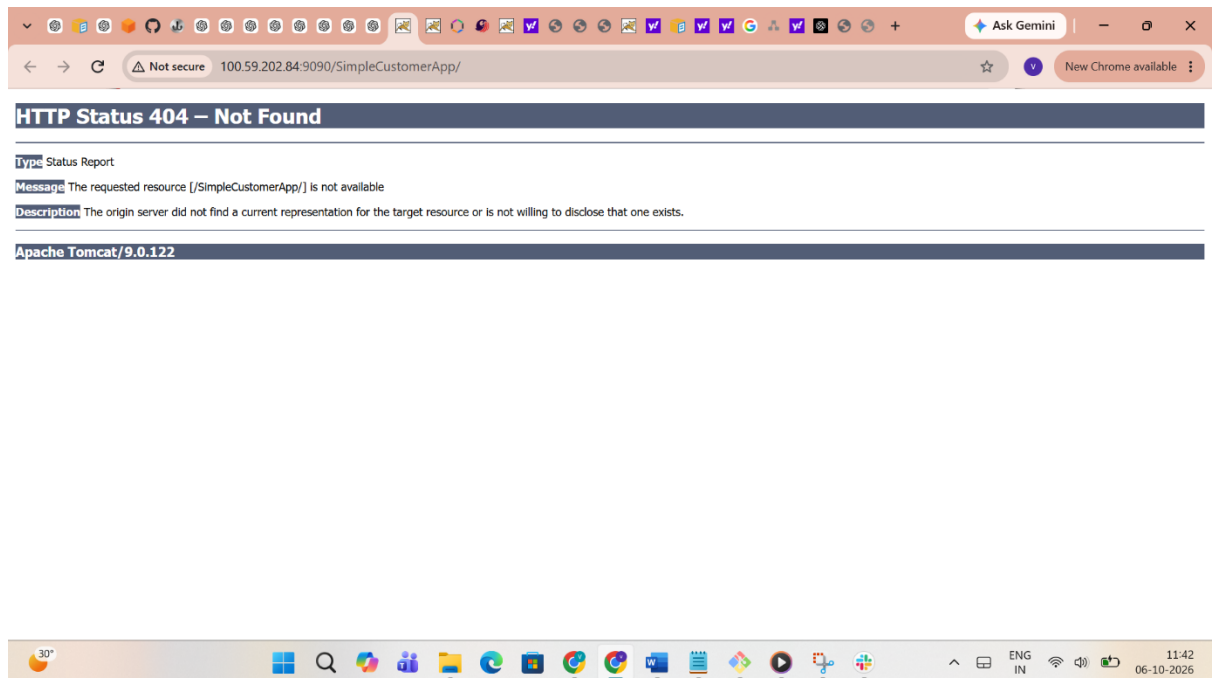
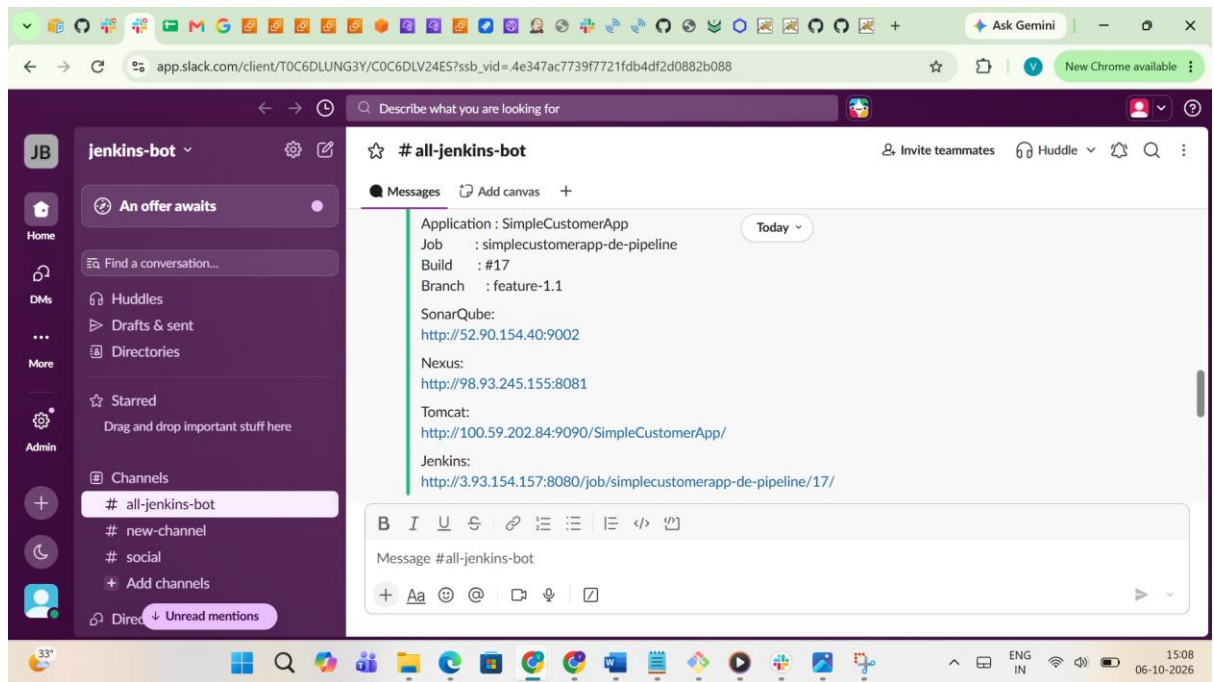
4-SNAPSHOT

5-SNAPSHOT

com/javatpoint/SimpleCustomerApp/15-SNAPSHOT

Delete folder

15:02 06-10-2026



3) Setup a Jenkins CICD pipeline using Scripted pipeline using feature-1.1 branch.

Source Code:

https://github.com/betawins/sabear_simplecutomerapp/tree/feature-1.1

stages:

- Git Clone

- **SonarQube Integration**
- **Maven Compilation**
- **Nexus Artifactory**
- **Slack Notification**
- **Deploy On tomcat**

Implementation Steps

1. Open Jenkins Dashboard.
2. Click New Item.
3. Enter:
simplecustomerapp-scri-pipeline
4. Select Pipeline.
5. Click OK.
6. Go to the Pipeline section.
7. Select Pipeline script.
8. Use a Scripted Pipeline beginning with:

node {

```
// =====
// TOOL CONFIGURATION
// =====
```

```
def mvnHome = tool name: 'MVN_HOME', type: 'maven'
```

```
// =====
// APPLICATION
// =====
```

```
def APP_NAME = 'SimpleCustomerApp'
```

```
// =====  
  
// GIT  
  
// =====  
  
def GIT_URL = 'https://github.com/betawins/sabear_simplecutomerapp.git'  
def GIT_BRANCH = 'feature-1.1'  
  
// =====  
  
// SONARQUBE  
  
// =====  
  
def SONAR_SERVER = 'sonarqube-server'  
def SONAR_URL = 'http://52.90.154.40:9002'  
  
// =====  
  
// NEXUS  
  
// =====  
  
def NEXUS_URL = '98.93.245.155:8081'  
def NEXUS_REPOSITORY = 'samplecustomer_app'  
def NEXUS_CREDENTIALS = 'nexus-cred'  
  
// =====  
  
// TOMCAT  
  
// =====  
  
def TOMCAT_URL = 'http://100.59.202.84:9090'  
def TOMCAT_CREDENTIALS = 'tomcat-cred'  
def TOMCAT_CONTEXT = 'SimpleCustomerApp'
```

```

// =====
// SLACK
// =====

def SLACK_CHANNEL = '#all-jenkins-bot'
def SLACK_CREDENTIALS = 'slack-cred'

// =====
// MAVEN ARTIFACT
// =====

def GROUP_ID = 'com.javatpoint'
def ARTIFACT_ID = 'SimpleCustomerApp'

/*
 * The original pom.xml contains:
 *
 * ${BUILD_NUMBER}-SNAPSHOT
 *
 * We cannot modify the original POM permanently.
 *
 * Therefore Jenkins creates:
 *
 * jenkins-build-pom.xml
 *
 * with:
 *
 * 1.0.0-SNAPSHOT

```

```
*/
```

```
def BUILD_VERSION = '1.0.0-SNAPSHOT'
```

```
// WAR will be detected dynamically
```

```
def WAR_FILE = "
```

```
try {
```

```
    // =====
```

```
    // MAVEN PATH
```

```
    // =====
```

```
    env.PATH = "${mvnHome}/bin:${env.PATH}"
```

```
    // =====
```

```
    // STAGE 1 - GIT CLONE
```

```
    // =====
```

```
    stage('Git Clone') {
```

```
        echo ""
```

```
        =====
```

```
        STAGE 1: GIT CLONE
```

```
        =====
```

```
    ""
```

```
        deleteDir()
```

```
git(  
    branch: GIT_BRANCH,  
    url: GIT_URL  
)  
  
sh ""  
    set -e  
  
    echo "Git clone completed successfully."  
  
    echo ""  
    echo "Current branch:"  
    git branch --show-current  
  
    echo ""  
    echo "Latest commit:"  
    git log -1 --oneline  
  
    echo ""  
    echo "Workspace:"  
    pwd  
  
    echo ""  
    echo "Repository files:"  
    ls -la  
    ""  
}
```

```

// =====
// STAGE 2 - PREPARE BUILD POM
// =====

stage('Prepare Build POM') {

    echo ""

    =====

    STAGE 2: PREPARE BUILD POM

    =====

    ""

    sh ""

        set -e

        echo "Original pom.xml:"
        cat pom.xml

        echo ""
        echo "Creating temporary Jenkins POM..."

        cp pom.xml jenkins-build-pom.xml

        echo ""
        echo "Replacing BUILD_NUMBER version..."

        sed -i 's#<version>\\${BUILD_NUMBER}-SNAPSHOT</version>#<version>1.0.0-SNAPSHOT</version>#g' jenkins-build-pom.xml

        echo ""

```

```
echo "Temporary POM version:"
```

```
grep -n '<version>' jenkins-build-pom.xml
```

```
echo ""
```

```
echo "Validating temporary POM..."
```

```
mvn -f jenkins-build-pom.xml validate
```

```
echo ""
```

```
echo "Temporary POM validation successful."
```

```
""
```

```
}
```

```
// =====
```

```
// STAGE 3 - MAVEN COMPILATION
```

```
// =====
```

```
stage('Maven Compilation') {
```

```
    echo ""
```

```
    =====
```

```
    STAGE 3: MAVEN COMPILATION
```

```
    =====
```

```
    ""
```

```
    sh ""
```

```
        set -e
```

```
echo "Maven version:"
```

```
mvn --version
```

```
echo ""
```

```
echo "Java version:"
```

```
java -version
```

```
echo ""
```

```
echo "Removing old target directory..."
```

```
rm -rf target
```

```
echo ""
```

```
echo "Building application..."
```

```
mvn -f jenkins-build-pom.xml clean package -DskipTests
```

```
echo ""
```

```
echo "Maven build completed successfully."
```

```
echo ""
```

```
echo "Target directory:"
```

```
ls -lah target/
```

```
echo ""
```

```
echo "Generated WAR files:"
```

```
find target -maxdepth 1 -type f -name "*.war" -print
```

```

'''

// Dynamically detect WAR

WAR_FILE = sh(
    script: '''
        find target -maxdepth 1 -type f -name "*.war" | head -1
    ''',
    returnStdout: true
).trim()

if (!WAR_FILE) {
    error("WAR file was not generated.")
}

echo "Detected WAR file: ${WAR_FILE}"
}

// =====
// STAGE 4 - SONARQUBE
// =====

stage('SonarQube Integration') {

    echo '''
=====
STAGE 4: SONARQUBE ANALYSIS
=====

```

'''

```
echo "SonarQube Jenkins server: ${SONAR_SERVER}"
```

```
echo "SonarQube URL: ${SONAR_URL}"
```

```
/*
```

```
* withSonarQubeEnv is still used so Jenkins can
```

```
* provide SonarQube authentication/configuration.
```

```
*
```

```
* However, sonar.host.url is explicitly supplied
```

```
* below using the known working URL.
```

```
*/
```

```
withSonarQubeEnv(SONAR_SERVER) {
```

```
    sh """
```

```
        set -e
```

```
        echo "Running SonarQube analysis..."
```

```
        echo ""
```

```
        echo "Explicit SonarQube URL:"
```

```
        echo "${SONAR_URL}"
```

```
        echo ""
```

```
        echo "Testing SonarQube server..."
```

```
        curl -f -s -o /dev/null "${SONAR_URL}/api/system/status"
```

```
echo "SonarQube server is reachable."
```

```
echo ""
```

```
echo "Executing Sonar Maven Plugin..."
```

```
mvn \
```

```
org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \
```

```
-Dsonar.projectKey=SimpleCustomerAppKey \
```

```
-Dsonar.projectName=SimpleCustomerApp \
```

```
-Dsonar.projectVersion=${env.BUILD_NUMBER} \
```

```
-Dsonar.host.url=${SONAR_URL}
```

```
echo ""
```

```
echo "SonarQube analysis completed successfully."
```

```
""
```

```
}
```

```
}
```

```
// =====
```

```
// STAGE 5 - VERIFY WAR
```

```
// =====
```

```
stage('Verify WAR') {
```

```
echo ""
```

```
=====
```

```
STAGE 5: VERIFY WAR
```

```
=====
```

'''

```
sh '''
```

```
set -e
```

```
echo "Detected WAR:"
```

```
echo "${WAR_FILE}"
```

```
if [ ! -f "${WAR_FILE}" ]; then
```

```
    echo ""
```

```
    echo "ERROR: WAR file was not found."
```

```
    echo ""
```

```
    echo "Files inside target:"
```

```
    ls -lah target/
```

```
    exit 1
```

```
fi
```

```
echo ""
```

```
echo "WAR file found successfully."
```

```
ls -lh "${WAR_FILE}"
```

```
echo ""
```

```
echo "WAR contents:"
```

```
unzip -l "${WAR_FILE}"
```

```

        echo ""

        echo "Checking application files..."

        unzip -l "${WAR_FILE}" | grep -E "index.html|Welcome.jsp|Customer.jsp|WEB-INF" || true

        echo ""

        echo "WAR verification completed successfully."
        ""
    }

// =====
// STAGE 6 - NEXUS
// =====

stage('Nexus Artifactory') {

    echo ""

    =====

    STAGE 6: NEXUS ARTIFACT UPLOAD

    =====

    ""

    sh ""

    set -e

    echo "Nexus URL:"

    echo "${NEXUS_URL}"

```

```
echo ""  
echo "Nexus repository:"  
echo "${NEXUS_REPOSITORY}"
```

```
echo ""  
echo "Artifact:"  
echo "${WAR_FILE}"
```

```
echo ""  
echo "Checking WAR..."
```

```
ls -lh "${WAR_FILE}"
```

```
""
```

```
nexusArtifactUploader(  
  
    nexusVersion: 'nexus3',  
  
    protocol: 'http',  
  
    nexusUrl: NEXUS_URL,  
  
    groupId: GROUP_ID,  
  
    version: BUILD_VERSION,  
  
    repository: NEXUS_REPOSITORY,
```

```
credentialsId: NEXUS_CREDENTIALS,
```

```
artifacts: [
```

```
  [
```

```
    artifactId: ARTIFACT_ID,
```

```
    classifier: "",
```

```
    file: WAR_FILE,
```

```
    type: 'war'
```

```
  ]
```

```
]
```

```
)
```

```
echo ""
```

```
=====
```

```
NEXUS UPLOAD SUCCESSFUL
```

```
=====
```

```
""
```

```
}
```

```
// =====
```

```
// STAGE 7 - TOMCAT DEPLOYMENT
```

```
// =====
```

```
stage('Deploy On Tomcat') {
```

```
echo ""
```

```
=====
```

```
STAGE 7: DEPLOY ON TOMCAT
```

```
=====
'''

    echo "Tomcat URL: ${TOMCAT_URL}"
    echo "WAR file: ${WAR_FILE}"
    echo "Context: ${TOMCAT_CONTEXT}"

    deploy(

        adapters: [
            tomcat9(
                credentialsId: TOMCAT_CREDENTIALS,
                url: TOMCAT_URL,
                path: "

            )
        ],

        contextPath: TOMCAT_CONTEXT,

        war: WAR_FILE
    )

    echo ""
=====

TOMCAT DEPLOYMENT COMPLETED

=====
'''

}
```

```

// =====
// STAGE 8 - VERIFY TOMCAT
// =====

stage('Verify Tomcat Deployment') {

    echo ""

    =====
    STAGE 8: VERIFY TOMCAT DEPLOYMENT
    =====
    ""

    sh ""

    set +e

    echo "Checking Tomcat Manager..."

    curl -u deployer:deployer \
    "$TOMCAT_URL/manager/text/list"

    echo ""

    echo "Checking application..."

    HTTP_STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
    "$TOMCAT_URL/$TOMCAT_CONTEXT/")

    echo ""

```

```
echo "Application HTTP status: $HTTP_STATUS"
```

```
echo ""
```

```
if [ "$HTTP_STATUS" = "200" ]; then
```

```
    echo "SimpleCustomerApp is responding successfully."
```

```
elif [ "$HTTP_STATUS" = "302" ]; then
```

```
    echo "SimpleCustomerApp is deployed and returned HTTP 302."
```

```
elif [ "$HTTP_STATUS" = "404" ]; then
```

```
    echo "Application is deployed but returned HTTP 404."
```

```
echo ""
```

```
echo "Checking Tomcat Manager application list..."
```

```
curl -u deployer:deployer \  
"$TOMCAT_URL/manager/text/list"
```

```
echo ""
```

```
echo "IMPORTANT:"
```

```
echo "Tomcat reports the application as deployed."
```

```
echo "HTTP 404 means the WAR does not expose a valid"
```

```
echo "welcome page or requested resource."
```

```

else

    echo "Application returned HTTP $HTTP_STATUS."

fi

echo ""
echo "Tomcat verification completed."
""
}

// =====
// STAGE 9 - SLACK
// =====

stage('Slack Notification') {

    echo ""

    =====
    STAGE 9: SLACK NOTIFICATION
    =====
    ""

    slackSend(

        channel: SLACK_CHANNEL,

        color: 'good',

```

tokenCredentialId: SLACK_CREDENTIALS,

botUser: true,

message: ""

PIPELINE SUCCESS

Application : \${APP_NAME}

Job : \${env.JOB_NAME}

Build : #\${env.BUILD_NUMBER}

Branch : \${GIT_BRANCH}

SonarQube:

\${SONAR_URL}

Nexus:

http://\${NEXUS_URL}

Repository:

\${NEXUS_REPOSITORY}

WAR:

\${WAR_FILE}

Tomcat:

\${TOMCAT_URL}/\${TOMCAT_CONTEXT}/

Jenkins:

```
${env.BUILD_URL}
```

```
""
```

```
)
```

```
}
```

```
// =====
```

```
// PIPELINE SUCCESS
```

```
// =====
```

```
echo ""
```

```
=====
```

```
PIPELINE SUCCESS
```

```
=====
```

```
""
```

```
echo "Application deployment completed."
```

```
echo ""
```

```
echo "Application URL:"
```

```
echo "${TOMCAT_URL}/${TOMCAT_CONTEXT}/"
```

```
} catch (err) {
```

```
// =====
```

```
// PIPELINE FAILURE
```

```
// =====
```

```
    echo ""

=====

PIPELINE FAILED

=====

""
```

```
    echo "Error: ${err}"
```

```
    slackSend(
```

```
        channel: SLACK_CHANNEL,
```

```
        color: 'danger',
```

```
        tokenCredentialId: SLACK_CREDENTIALS,
```

```
        botUser: true,
```

```
        message: ""
```

```
PIPELINE FAILED
```

```
Application : ${APP_NAME}
```

```
Job      : ${env.JOB_NAME}
```

```
Build    : #${env.BUILD_NUMBER}
```

```
Branch   : ${GIT_BRANCH}
```

```
Error:
```

```
${err}
```

Please check Jenkins Console Output.

Jenkins:

```
${env.BUILD_URL}
```

```
""
```

```
)
```

```
throw err
```

```
} finally {
```

```
echo ""
```

```
=====
```

```
PIPELINE EXECUTION COMPLETED
```

```
=====
```

```
""
```

```
// Workspace intentionally retained for troubleshooting.
```

```
}
```

```
}
```

9. Configure the Maven tool:

```
MVN_HOME
```

10. Configure the Git repository:

```
https://github.com/betawins/sabear_simplecutomerapp.git
```

11. Configure:

```
Branch: feature-1.1
```

12. Configure SonarQube server.

13. Configure Nexus credentials.

14. Configure Slack credentials.

15. Configure Tomcat credentials.

16. Create the following scripted stages:

Git Clone



Maven Compilation



SonarQube Integration



Nexus Artifactory



Slack Notification



Deploy On Tomcat

17. Use Jenkins stage() blocks for each stage.

18. Run Maven to generate the WAR.

19. Detect/verify the generated WAR file.

20. Run SonarQube analysis.

21. Upload the WAR to Nexus.

22. Send a Slack notification.

23. Deploy the WAR to Tomcat.

24. Add try/catch/finally handling if required.

25. Send a failure notification from the catch block.

26. Click Save.

27. Click Build Now.

28. Verify that all stages execute successfully.

29. Verify the application from Tomcat.

simplecustomerapp-scri-pipeline | hiring_app Config - Jenkins | Tools - Manage Jenkins - Jenkins | HTTP Status 404 - Not Found | + | - | Incognito | New Chrome available

Not secure 3.93.154.157:8080/job/simplecustomerapp-scri-pipeline/configure

Jenkins / simplecustomerapp-scri-p... / Configure

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

```
1 node {
2
3 // =====
4 // TOOL CONFIGURATION
5 // =====
6
7 def mvnHome = tool name: 'MVN_HOME', type: 'maven'
8
9 // =====
10 // APPLICATION
11 // =====
12
```

Save Apply

33° 14:57 06-10-2026

simplecustomerapp-scri-pipeline | hiring_app Config - Jenkins | Tools - Manage Jenkins - Jenkins | HTTP Status 404 - Not Found | + | - | Incognito | New Chrome available

Not secure 3.93.154.157:8080/job/simplecustomerapp-scri-pipeline/11/

Jenkins / simplecustomerapp-scri-p... / #11

Status **#11 (Oct 6, 2026, 6:09:52 AM)** Add description Keep this build forever

Changes

Console Output

Delete build '#11'

Git Build Data

Pipeline Overview

Edit build information

Replay

Pipeline Steps

Workspaces

Previous Build

Started by user vamshikumar

Revision: 05222fef4b498f40dd73a8a1b6fbdb0c619d398a

Repository: https://github.com/betawins/sabear_simplecutomerapp.git

- refs/remotes/origin/feature-1.1

No changes.

Started 3 hr 18 min ago
Took 17 sec

REST API Jenkins 2.580.1

33° 14:58 06-10-2026

simplecustomerapp-scri-pipeline | hiring_app Config - Jenkins | Tools - Manage Jenkins - Jenkins | HTTP Status 404 - Not Found | + | Not secure | 3.93.154.157:8080/job/simplecustomerapp-scri-pipeline/11/console | Incognito | New Chrome available

Jenkins / simplecustomerapp-scri-p... / #11

```
Application deployment completed.
[Pipeline] echo

[Pipeline] echo
Application URL:
[Pipeline] echo
http://100.59.202.84:9090/SimpleCustomerApp/
[Pipeline] echo

=====
PIPELINE EXECUTION COMPLETED
=====

[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Jenkins 2.580.1

33° | Windows Taskbar | 14:58 06-10-2026

52.90.154.40:9002/projects | Ask Gemini | New Chrome available

Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine.

There's a new version of SonarQube Server available. Upgrade to SonarQube Server and get access to enterprise features. [Learn More](#)

The way in which security, reliability, and maintainability counts and ratings are calculated has changed. [Learn more in SonarQube documentation](#)

SonarQube community | Projects | Issues | Rules | Quality profiles | Quality gates | Administration | More

My Favorites | All

Filters

Quality gate

✓ Passed 3 | ✗ Failed 0

Security

A ≥ 0 info issues 2 | B ≥ 1 low issue 1

SimpleCustomerApp Public ✓ Passed

Last analysis: 2 hours ago - 28 Lines of Code - XML

A 0	A 0	A 0	A —	—	0.0%
Security	Reliability	Maintainability	Hotspots Reviewed	Coverage	Duplications

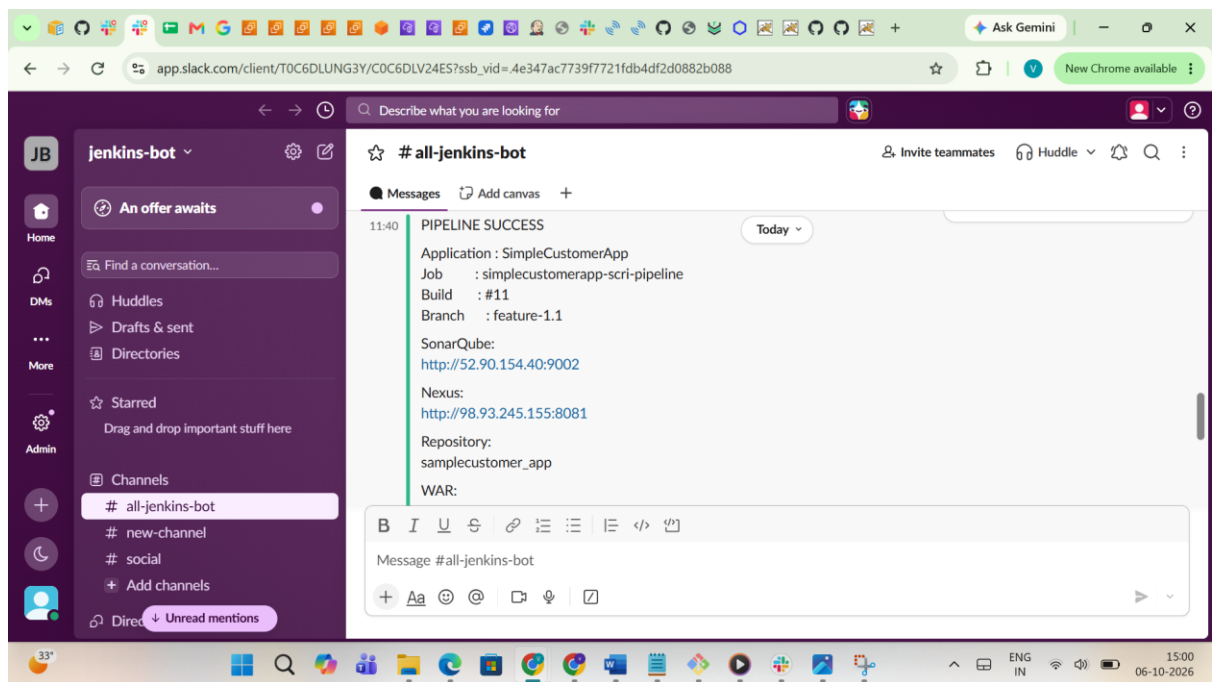
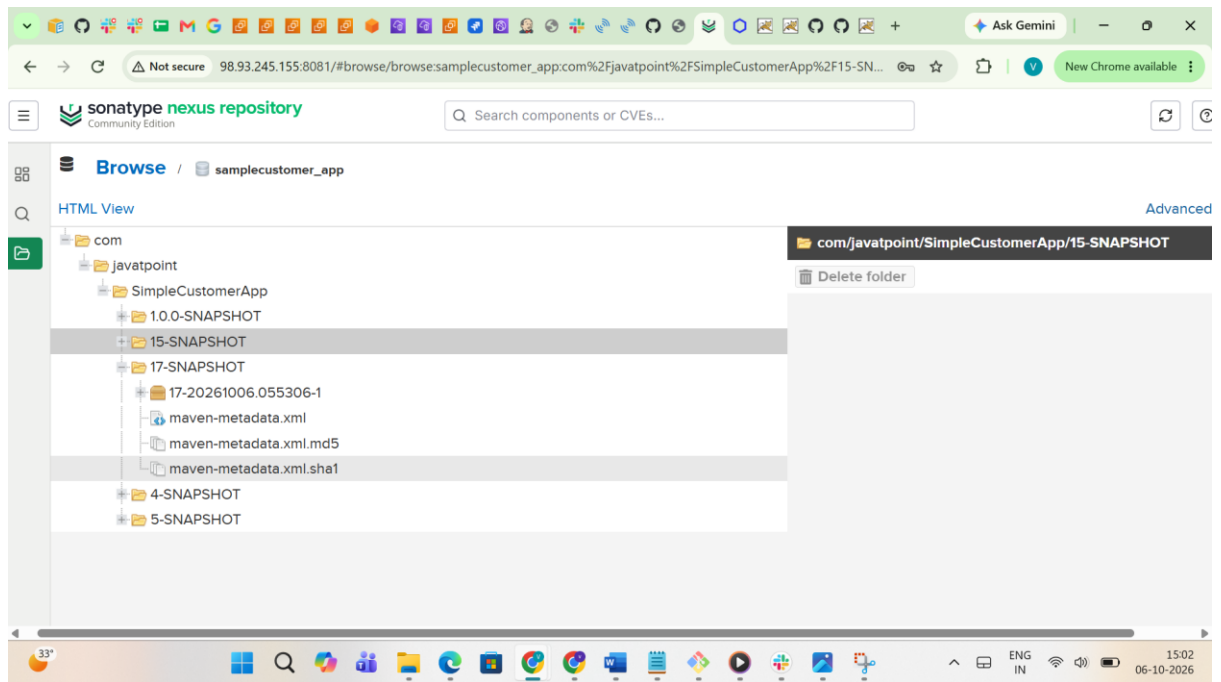
3 of 3 shown

SonarQube™ technology is powered by SonarSource Sàrl

Community Build v26.9.0.129388 - MQR MODE

LGPL v3 | Community | Documentation | Plugins | Web API

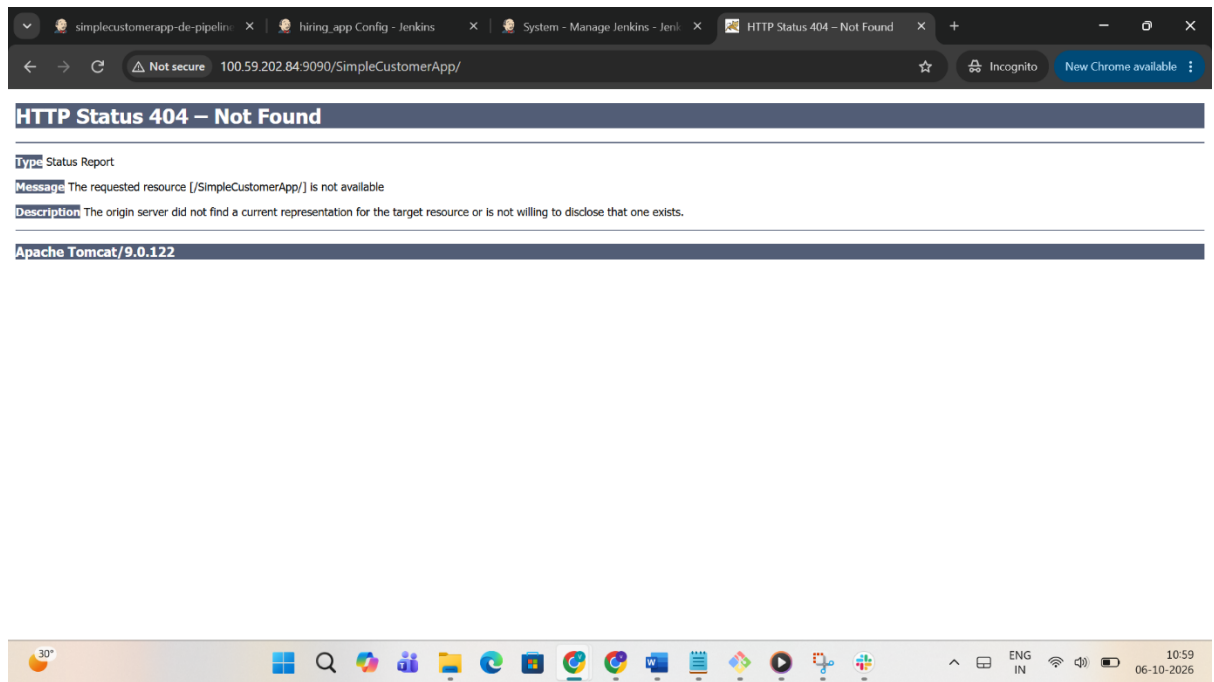
33° | Windows Taskbar | 14:57 06-10-2026



WAR:
target/SimpleCustomerApp-1.0.0-SNAPSHOT.war

Tomcat:
<http://100.59.202.84:9090/SimpleCustomerApp/>

Jenkins:
<http://3.93.154.157:8080/job/simplecustomerapp-scri-pipeline/11/>



4) Write sample skeleton of pipelines.

Implementation Steps

1. basic Declarative Pipeline skeleton.

Declarative Pipeline Skeleton

```
pipeline {  
    agent any  
  
    stages {  
  
        stage('Git Clone') {  
            steps {  
                echo 'Cloning source code'  
            }  
        }  
  
        stage('Maven Compilation') {  
            steps {
```

```
        echo 'Compiling application'
    }
}

stage('SonarQube') {
    steps {
        echo 'Running SonarQube analysis'
    }
}

stage('Nexus') {
    steps {
        echo 'Uploading artifact'
    }
}

stage('Slack') {
    steps {
        echo 'Sending Slack notification'
    }
}

stage('Tomcat Deployment') {
    steps {
        echo 'Deploying application'
    }
}
}
```

Scripted Pipeline Skeleton

```
node {  
  
    stage('Git Clone') {  
        echo 'Cloning source code'  
    }  
  
    stage('Maven Compilation') {  
        echo 'Compiling application'  
    }  
  
    stage('SonarQube') {  
        echo 'Running SonarQube analysis'  
    }  
  
    stage('Nexus') {  
        echo 'Uploading artifact'  
    }  
  
    stage('Slack') {  
        echo 'Sending Slack notification'  
    }  
  
    stage('Tomcat Deployment') {  
        echo 'Deploying application'  
    }  
}
```

Pipeline Flow

Git

↓

Maven

↓

SonarQube

↓

Nexus

↓

Slack

↓

Tomcat

5) Create a parameterized job in Jenkins.

Source Code: <https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git>

Implementation Steps

1. Open Jenkins Dashboard.
2. Click New Item.
3. Enter:
spring-mvc-parameterized-job
4. Select Freestyle project.
5. Click OK.
6. Enable:
This project is parameterized
7. Click Add Parameter.
8. Select Choice Parameter.
9. Enter:
Name: BUILD_TYPE
10. Add the following choices:
FULL_BUILD
CLEAN_BUILD

11. Description:

Select the Maven build type.

Select Choice Parameter.

Enter:

Name: ENVIRONMENT

Add the following choices:

DEV

QA

PROD

Description:

Select the Environment type.

12. Go to Source Code Management.

13. Select Git.

14. Enter:

<https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git>

15. Set the branch:

*/master

16. Configure Maven under Manage Jenkins → Tools.

17. If Maven is manually installed, configure the correct Maven home, for example:

/opt/maven

18. Go to Build Steps.

19. Select Execute shell.

20. Add:

#!/bin/bash

```
echo "====="
```

```
echo "PARAMETERIZED JENKINS JOB"
```

```
echo "====="
```

```
echo ""  
echo "Selected Environment : $ENVIRONMENT"  
echo "Selected Build Type : $BUILD_TYPE"  
echo "Build Number      : $BUILD_NUMBER"  
echo "Job Name          : $JOB_NAME"  
echo ""
```

```
echo "Checking workspace..."  
pwd
```

```
echo ""  
echo "Repository files:"  
ls -la
```

```
echo ""  
echo "===== "  
echo "BUILD TYPE"  
echo "===== "
```

```
if [ "$BUILD_TYPE" = "CLEAN_BUILD" ]; then
```

```
    echo "Performing CLEAN BUILD..."
```

```
    echo "Removing previous build directory..."
```

```
    rm -rf build-output
```

```
    mkdir -p build-output
```

```
echo "Clean build directory created."
```

```
else
```

```
echo "Performing FULL BUILD..."
```

```
mkdir -p build-output
```

```
echo "Full build directory created."
```

```
fi
```

```
echo ""
```

```
echo "=====
```

```
echo "ENVIRONMENT"
```

```
echo "=====
```

```
if [ "$ENVIRONMENT" = "DEV" ]; then
```

```
echo "Deploying configuration for DEV environment."
```

```
elif [ "$ENVIRONMENT" = "QA" ]; then
```

```
echo "Deploying configuration for QA environment."
```

```
elif [ "$ENVIRONMENT" = "PROD" ]; then
```

```
echo "Deploying configuration for PROD environment."
```

fi

echo ""

echo "Creating build information..."

Application: Spring MVC Application

Environment: \$ENVIRONMENT

Build Type: \$BUILD_TYPE

Build Number: \$BUILD_NUMBER

Job Name: \$JOB_NAME

Build URL: \$BUILD_URL

EOF

echo ""

echo "Build information:"

cat build-output/build-info.txt

echo ""

echo "=====

echo "PARAMETERIZED BUILD SUCCESSFUL"

echo "=====

21. Click Apply.

22. Click Save.

23. Initially, Jenkins may show Build with Parameters after the parameterized configuration is saved.

24. Click Build with Parameters.

25. Select:

FULL_BUILD

or:

CLEAN_BUILD

And

DEV

OR

QA

OR

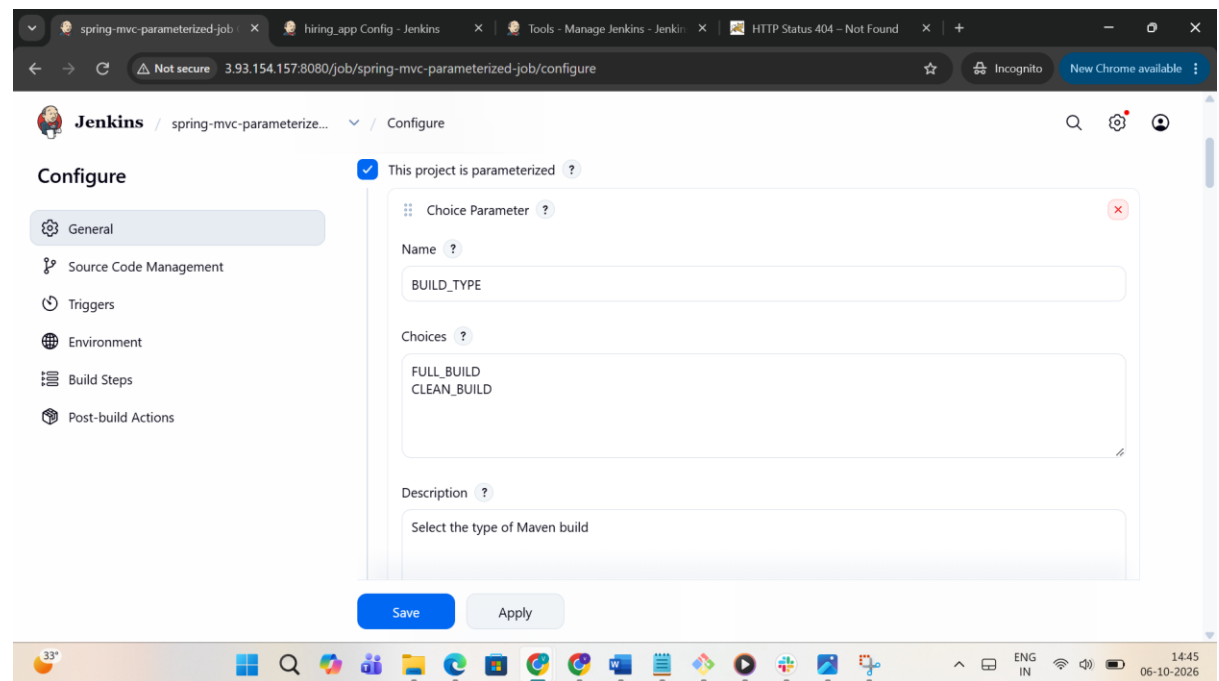
PROD

26. Click Build.

27. Jenkins checks out the master branch.

28. Jenkins reads the selected BUILD_TYPE and ENVIRONMENT.

31. Check Console Output to verify the selected parameter and Maven execution.



spring-mvc-parameterized-job hiring_app Config - Jenkins Tools - Manage Jenkins - Jenkins HTTP Status 404 - Not Found

Not secure 3.93.154.157:8080/job/spring-mvc-parameterized-job/configure

Jenkins / spring-mvc-parameterize... / Configure

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

Choice Parameter ?

Name ?

ENVIRONMENT

Choices ?

DEV
QA
PROD

Description ?

Select deployment environment

Save Apply

33° 14:46 06-10-2026

spring-mvc-parameterized-job hiring_app Config - Jenkins Tools - Manage Jenkins - Jenkins HTTP Status 404 - Not Found

Not secure 3.93.154.157:8080/job/spring-mvc-parameterized-job/configure

Jenkins / spring-mvc-parameterize... / Configure

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

Execute shell ?

Command

See the list of available environment variables

```
#!/bin/bash

echo "=====
echo "PARAMETERIZED JENKINS JOB"
echo "=====

echo ""
echo "Selected Environment : $ENVIRONMENT"
echo "Selected Build Type : $BUILD_TYPE"
echo "Build Number : $BUILD_NUMBER"
echo "Job Name : $JOB_NAME"
echo ""

echo "Checking workspace..."
pwd
```

Save Apply

33° 14:47 06-10-2026

Jenkins / spring-mvc-parameterize...

Delete Project FULL_BUILD

Configure

Rename

Builds > Filter

Today

- #9 6:50 AM
- #8 6:49 AM
- #7 6:48 AM
- #6 6:48 AM
- #5 6:44 AM
- #4 6:40 AM
- #3 6:36 AM

ENVIRONMENT

Select deployment environment

DEV

DEV

QA

PROD

Jenkins / spring-mvc-parameterize...

Status

Changes

Workspace

Build with Parameters

Delete Project

Configure

Rename

Builds > Filter

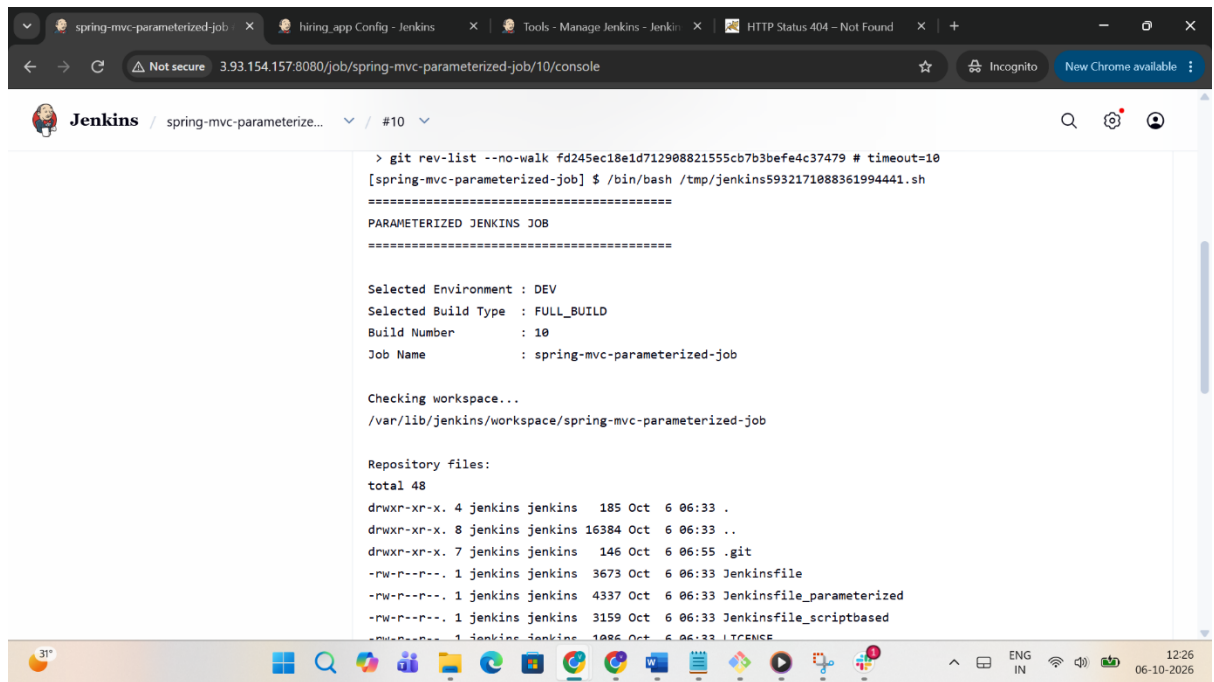
Today

- #10 6:55 AM
- #9 6:50 AM
- #8 6:49 AM

spring-mvc-parameterized-job

Permalinks

- Last build (#10), 2 hr 19 min ago
- Last stable build (#10), 2 hr 19 min ago
- Last successful build (#10), 2 hr 19 min ago
- Last failed build (#9), 2 hr 23 min ago
- Last unsuccessful build (#9), 2 hr 23 min ago
- Last completed build (#10), 2 hr 19 min ago

A screenshot of a web browser displaying the Jenkins console output for a job named 'spring-mvc-parameterized-job'. The browser's address bar shows the URL '3.93.154.157:8080/job/spring-mvc-parameterized-job/10/console'. The Jenkins interface shows the job name and build number '#10'. The console output displays the following text:

```
> git rev-list --no-walk fd245ec18e1d712908821555cb7b3befe4c37479 # timeout=10
[spring-mvc-parameterized-job] $ /bin/bash /tmp/jenkins5932171088361994441.sh
=====
PARAMETERIZED_JENKINS_JOB
=====

Selected Environment : DEV
Selected Build Type : FULL_BUILD
Build Number : 10
Job Name : spring-mvc-parameterized-job

Checking workspace...
/var/lib/jenkins/workspace/spring-mvc-parameterized-job

Repository files:
total 48
drwxr-xr-x. 4 jenkins jenkins 185 Oct 6 06:33 .
drwxr-xr-x. 8 jenkins jenkins 16384 Oct 6 06:33 ..
drwxr-xr-x. 7 jenkins jenkins 146 Oct 6 06:55 .git
-rw-r--r--. 1 jenkins jenkins 3673 Oct 6 06:33 Jenkinsfile
-rw-r--r--. 1 jenkins jenkins 4337 Oct 6 06:33 Jenkinsfile_parameterized
-rw-r--r--. 1 jenkins jenkins 3159 Oct 6 06:33 Jenkinsfile_scriptbased
-rw-r--r--. 1 jenkins jenkins 1086 Oct 6 06:33 LICENSE
```

6) Setup one slave machine for Jenkins.

Implementation steps:

1. Login to the **Slave-01** machine.
2. Switch to root user: `sudo su -`
3. Install JDK 11: `sudo yum install java-21-amazon-corretto -y`
4. Verify Java: `java -version`
5. Generate SSH key: `ssh-keygen`
6. Navigate to the SSH directory: `cd ~/.ssh`
7. Add the public key to authorized_keys: `cat id_rsa.pub >> authorized_keys`
8. Set permissions: `chmod 700 authorized_keys`

Steps on Jenkins Master

10. Login to the Jenkins Master.
11. Switch to root: `sudo su -`
12. Create the Jenkins SSH directory: `mkdir -p /var/lib/jenkins/.ssh`
13. Navigate to the directory: `cd /var/lib/jenkins/.ssh`
14. Add Slave-01 to known_hosts: `ssh-keyscan -H <SLAVE-01-IP> >> /var/lib/jenkins/.ssh/known_hosts`

15. Add Slave-02 to known_hosts: `ssh-keyscan -H <SLAVE-02-IP> >> /var/lib/jenkins/.ssh/known_hosts`

16. Change ownership: `chown jenkins:jenkins known_hosts`

17. Set permissions: `chmod 700 known_hosts`

Steps in Jenkins GUI

18. Login to Jenkins.

19. Go to **Manage Jenkins** → **Nodes**.

20. Click **New Node**.

21. Enter the node name as **Slave-01**.

22. Select **Permanent Agent** and click **Create**.

23. Enter the remote root directory: `/home/ec2-user`

24. Set the number of executors to **2**.

25. Enter the label: **Java**

26. Under **Launch method**, select **Launch agents via SSH**.

27. Enter the **public IP of Slave-01**.

28. Under **Credentials**, select **SSH Username with private key**.

29. Enter the username: `ec2-user`

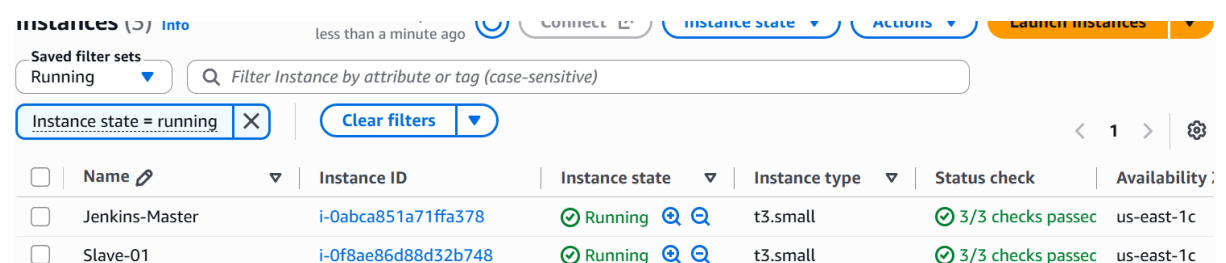
30. Paste the EC2 `.pem` private key.

31. Select **Known hosts file** as the host key verification strategy.

32. Click **Save**.

33. Verify that **Slave-01** is **Online**.

34. Jenkins Master can now execute jobs on slave machine.



Instances (2) Info							less than a minute ago	Connect	Instance state	Actions	Launch instances
Saved filter sets		Running					Filter Instance by attribute or tag (case-sensitive)				
Instance state = running		Clear filters					< 1 > ⚙				
☐	Name	Instance ID	Instance state	Instance type	Status check	Availability					
☐	Jenkins-Master	i-0abca851a71ffa378	Running	t3.small	3/3 checks passec	us-east-1c					
☐	Slave-01	i-0f8ae86d88d32b748	Running	t3.small	3/3 checks passec	us-east-1c					

```
[root@ip-172-31-26-204 ~]# java -version
openjdk version "21.0.12.1" 2026-08-18 LTS
OpenJDK Runtime Environment Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS, mixed mode, sharing)
[root@ip-172-31-26-204 ~]# ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Enter passphrase for "/root/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:DKgeMlEZGFbvZMAOPGSFPLgwiBGyxiTtDLgxJ+t4hQ root@ip-172-31-26-204.ec2.internal
The key's randomart image is:
+---[RSA 3072]-----+
| ^%B* |
| ^%o.o. |
| *B* ++. |
| +E =+. o |
| o +. . S |
| * o |
| o o |
+-----+

```

i-Of8ae86d88d32b748 (Slave-01)

```

| . |
+----[SHA256]-----+
[root@ip-172-31-26-204 ~]# cd .ssh/
[root@ip-172-31-26-204 .ssh]# ls
authorized_keys id_rsa id_rsa.pub
[root@ip-172-31-26-204 .ssh]# cat id_rsa.pub > authorized_keys
[root@ip-172-31-26-204 .ssh]# cat authorized_keys
ssh-rsa AAAAB3NzaClyc2EAAAADAQABAgQCoegE/Ob4iccnz6ZIVqBVdem2KA1gq3Y62AeEATxQdHQscnHJXmgBF4XF2JoXwZhg2xGARjbptjnlRiueXlgl
90v24iAIIAKq/ZnFGImAnSfdd60IX/K0siSpLFwYk9ctqhrbrbc5uxwOLDwflvN/jCc+iD+XnRa0uIG+ywjn/YhZ9iM7+wxEn25+gbFg0N5u4eFg8m68ckGICZed
QZzvUBGdsQSqXEW8ao7dokkHEA50ASPXTsiQvgfsRPOwVg0JO74D7x8nufA4hDtZmwyc/vsn4QDzrDbFsz9k5kPQNA1lnuqXYZJytrueIzRC2irrmKdFM0V7HkiY
pzhEl+beUXwh92I1jM/Istc/QTtcOiEMr6WA6ggotS9zrIDO8F+QHSQLTDbrBo7ppnsAcIiQkYfGhvprrm95QFArkum7ULDAaqlUEojD4ZiYxuVW5hvPfV0CPwfyh
-172-31-26-204.ec2.internal
[root@ip-172-31-26-204 .ssh]# chmod 700 authorized_keys
[root@ip-172-31-26-204 .ssh]#

```

i-Of8ae86d88d32b748 (Slave-01)

```
[root@ip-172-31-16-190 jenkins]# cd .ssh/
[root@ip-172-31-16-190 .ssh]# ls
[root@ip-172-31-16-190 .ssh]# ls -la
.
..
[root@ip-172-31-16-190 .ssh]# ssh-keyscan -H 54.144.48.173 >> /var/lib/jenkins/.ssh/known_hosts
[root@ip-172-31-16-190 .ssh]# ll
total 4
-rw-r--r--. 1 root root 559 Sep 30 07:32 known_hosts
[root@ip-172-31-16-190 .ssh]# chown jenkins:jenkins known_hosts
[root@ip-172-31-16-190 .ssh]# ll
total 4
-rw-r--r--. 1 jenkins jenkins 559 Sep 30 07:32 known_hosts
[root@ip-172-31-16-190 .ssh]# chmod 700 known_hosts
[root@ip-172-31-16-190 .ssh]# ll
total 4
-rwx-----. 1 jenkins jenkins 559 Sep 30 07:32 known_hosts
[root@ip-172-31-16-190 .ssh]#

```

i-0abca851a71ffa378 (Jenkins-Master)

Public IP: 54.144.48.173 Private IP: 172.31.16.190

```
[root@ip-172-31-16-190 jenkins]# mkdir .ssh
[root@ip-172-31-16-190 jenkins]# ls
config.xml                               jobs
hudson.model.UpdateCenter.xml            nodeMonitors.xml
hudson.plugins.emailext.ExtendedEmailPublisher.xml  org.jenkinsci.plugins.gitclient.GitHostKeyVerificationConfiguration
hudson.plugins.git.GitTool.xml           plugins
identity.key.enc                         queue.xml.bak
io.jenkins.plugins.primus.PrismConfiguration.xml  scriptApproval.xml
jenkins.install.InstallUtil.lastExecVersion  secret.key
jenkins.install.UpgradeWizard.state        secret.key.not-so-secret
jenkins.model.JenkinsLocationConfiguration.xml  secrets
jenkins.plugins.git.GitHooksConfiguration.xml  updates
jenkins.security.QueueItemAuthenticatorConfiguration.xml  userContent
jenkins.security.UpdateSiteWarningsConfiguration.xml  users
jenkins.security.apitoken.ApiTokenPropertyConfiguration.xml  workspace
jenkins.telemetry.Correlator.xml
[root@ip-172-31-16-190 jenkins]# ls -la
.
..
.cache
.groovy
.java
.lastStarted
.owner
jenkins.security.QueueItemAuthenticatorConfiguration.xml
jenkins.security.UpdateSiteWarningsConfiguration.xml
jenkins.security.apitoken.ApiTokenPropertyConfiguration.xml
jenkins.telemetry.Correlator.xml
jobs
nodeMonitors.xml
org.jenkinsci.plugins.gitclient.GitHostKeyVerificationConfiguration.xml

```

i-0abca851a71ffa378 (Jenkins-Master)

Jenkins / Manage Jenkins / Nodes

Launch agents via SSH

Host ?
54.144.48.173

Credentials ?
- none -

The selected credentials cannot be used for this node.

Host Key Verification Strategy ?
Known hosts file Verification Strategy

Advanced

Availability ?

< Add SSH Username with private key

ID ?
Slave-01

Description ?

Username
ec2-user

☐ Treat username as secret ?

Private Key
☒ Enter directly
☐ Key

Create

Jenkins / Manage Jenkins / Nodes

Use this node as much as possible

Launch method ?
Launch agents via SSH

Host ?
54.144.48.173

Credentials ?
ec2-user

Host Key Verification Strategy ?
Known hosts file Verification Strategy

Save

Jenkins

+ New Item

Build History

Project Relationship

Check File Fingerprint

Build Queue

No builds in the queue.

Build Executor Status

Built-In Node 0/2

Slave-01 0/2

Add description

All +

S	W	Name	Last Success	Last Failure	Last Duration
✓	☀	first_job	16 hr #1	N/A	15 sec

Icon: S M L

